Thought教AI思考

- 来源: https://class.imooc.com/lesson/5262#mid=104897
- 阶段: 阶段一: 【认知基石】理解大模型的本质与工作原理
- 周: 第3周 语言模型的进阶能力
- 模块: Context Learning
- 课节: 图文 1-4 Thought教AI思考

1-4 Thought教AI思考

Thought教AI思考

> [!TIP] 阅读指南 上一节，我们看了情境学习——通过在提示词中给几个例子，模型就能学会新任务。 但这里有一个更神奇的现象： 如果不仅给模型"答案"，还给"推理过程"，它的表现会大幅提升。 而且这个提升不是一点点，在某些复杂任务上，准确率可以从17%跃升到78%。 这个方法叫做CoT（Chain-of-Thought Prompting），中文可以译为"思维链提示"或"推理链提示"。 第一次看到这个结果时，很多研究者都感到震惊：为什么让模型"展示思考过程"会让它变得更聪明？ 让我带你深入这个现象。 本文中的错误示例基于早期的GPT-3。现在大模型普遍都内置了CoT，所以并不一定可以复现。

8.1 什么是思维链

思维链（Chain-of-Thought, CoT）：在提示词中不仅提供问题和答案，还提供从问题到答案的推理步骤，引导模型进行逐步推理。

核心思想是对比两种方式：

> 标准方法： 问题 → 答案 思维链： 问题 → 推理步骤1 → 推理步骤2 → ... → 答案

思维链就是教AI"展示你的工作过程"（Show your work）。

8.2 一个对比实验：加不加推理，差别有多大

用小学数学题来展示思维链的威力：

> 问题： 咖啡店有23杯咖啡。上午卖了7杯，下午又做了5杯，现在有多少杯？

问题：

不带推理过程

通过Few-Shot给出示例，但只有答案，没有推理。

> 例1： 问题：小明有15个苹果，吃了3个，还剩多少？ 答案：12个 例2： 问题：教室里有8个学生，又来了5个，现在有多少学生？ 答案：13个 例3（目标问题）： 问题：咖啡店有23杯咖啡。上午卖了7杯，下午又做了5杯，现在有多少杯？ 答案：

GPT-3的回答:

> 20个

错误！ 正确答案是 23 - 7 + 5 = 21。

错误！

当然，不是每次都会计算错误，这是一个概率问题。但在复杂数学题上，这种方法准确率只有17%左右。

带推理过程

同样用Few-Shot，但示例中包含完整推理步骤：

> 例1： 问题：小明有15个苹果，吃了3个，还剩多少？ 推理：小明原来有15个苹果，吃了3个，所以用减法：15 - 3 = 12 答案：12个 例2： 问题：教室里有8个学生，又来了5个，现在有多少学生？ 推理：原来8个学生，又来了5个，所以用加法：8 + 5 = 13 答案：13个 例3（目标问题）： 问题：咖啡店有23杯咖啡。上午卖了7杯，下午又做了5杯，现在有多少杯？ 推理：

GPT-3的回答:

> 推理：咖啡店原来有23杯，上午卖了7杯，剩下 23-7=16杯。 下午又做了5杯，所以 16+5=21杯。 答案：21杯

完全正确！不仅答案对了，推理步骤也清晰明了。

准确率从17%提升到57%（加上其他优化可达74%）。

17%提升到57%

> [!NOTE] 数据来源：Wei等，2022年，《Chain-of-Thought Prompting Elicits Reasoning in Large Language Models》

为什么示例中要有推理?

因为模型会"模仿"示例格式——示例有推理，它就会生成推理；示例只有答案，它就只给答案。

8.3 思维链的两种主要形式

研究者们发现，思维链有两种主要使用方式。

形式1：Few-Shot CoT（需要示例）

这就是刚才看到的方式——在示例中展示推理步骤，让模型模仿。

结构模板:

> 示例1: [问题] → [推理过程] → [答案] 示例2: [问题] → [推理过程] → [答案] ... 目标问题: [问题] → [模型自动生成推理] → [模型给出答案]

适用场景:

- 需要特定推理模式(如数学证明、逻辑推理)
- 想控制推理的风格和详细程度
- 有高质量示例可供参考

形式2：Zero-Shot CoT（不需要示例）

2022年的一个惊人发现：

不需要任何示例，只需在问题后加一句"Let’s think step by step"（让我们一步步思考），模型就会自动展示推理！

用同样的咖啡店问题测试：

> 咖啡店有23杯咖啡。上午卖了7杯，下午又做了5杯，现在有多少杯？ Let’s think step by step.

> 好的，让我们一步步来： 第一步：咖啡店原来有23杯咖啡 第二步：上午卖了7杯，剩下 23 - 7 = 16杯 第三步：下午又做了5杯，总共 16 + 5 = 21杯 因此，现在有21杯咖啡。

完全正确！ 仅仅加了一句话，准确率就有巨大提升。

完全正确！

常用触发短语:

英文：

- “Let’s think step by step”
- “Let’s approach this step-by-step”
- “Let’s break this down”

中文：

- “让我们一步步思考”
- “让我们仔细分析”
- “让我们分步骤解决”

为什么如此简单的一句话就有效?

预训练数据中有大量这样的文本模式:

> “让我们一步步解决这个问题: 第一步。... 第二步。... 因此。...”

模型学会了:看到"一步步"这类短语 → 应该生成详细推理过程。

Zero-Shot CoT巧妙利用了这个模式，无需示例就能触发推理能力。

8.4 思维链为什么有效：深层原理

这是最令人着迷的问题：为什么展示推理过程会让模型变得更"聪明"？

为什么展示推理过程会让模型变得更"聪明"？

分解复杂问题

复杂问题直接求解很难，但如果分解成小步骤，每一步都不难。

> 直接问： “23杯咖啡，上午卖7杯，下午做5杯，现在有多少？” 模型需要"一次性"推理： 23 - 7 + 5 = ? 这对模型的"工作记忆"要求很高 用思维链： 步骤1：23 - 7 = ? → 16 步骤2：16 + 5 = ? → 21 每一步都是简单的算术，容易做对

类比：

> 人类也是这样: 直接算 23 × 47 很难 但如果分解: 23 × 47 ——— 161 (23 × 7) 920 (23 × 40) ——— 1081 每一步都简单，组合起来解决复杂问题

利用自回归的优势

ChatGPT是自回归模型，一个词一个词生成。

> 问题 → 直接生成答案 模型只有一次机会，必须"想好了"才输出

思维链方法：

> 问题 → 生成推理步骤1 → 步骤2 → 步骤3 → 答案 ↑↑↑ 每生成一个词，都可以作为后续推理的"输入"

模型可以"边想边说"，前面的推理可以帮助后面的推理

一个具体的例子:

> 问题：“小明比小红大3岁，小红8岁，小明多少岁?”

如果直接生成答案:

> 模型需要在生成"11"之前，就在"脑海中"完成 8+3 的计算

如果用CoT:

> 模型生成：“小红8岁” → 这个信息现在在上下文中了 然后生成：“小明比小红大3岁” → 关系也明确了 然后生成：“所以8+3=11” → 可以"看着"前面的信息计算 最后生成：“小明11岁”

每一步都基于已经生成的内容，降低了认知负荷。

自我纠错机会

在推理过程中，模型有机会"发现"并"纠正"错误。

> 问题：“如果今天是周三，3天后是星期几?”

直接答:

> 可能错误地回答"周五"(算错了)

用CoT:

> “今天是周三， 1天后是周四， 2天后是周五， 3天后是周六” ← 在生成过程中逐步推进，不容易出错

或者即使中途出错:

> “今天是周三， 1天后是周四， 2天后是周五， 3天后是。...等等，应该是周六，不是周五”

模型在生成过程中可以自我纠正。

> [!NOTE] 如果你足够仔细看过DeepSeek等支持推理的大模型的推理过程，你会发现，它有时候确实会出现： "等等，这里好像有问题。让我修正一下。" 这其实就是模型在自我纠错。

8.5 思维链的局限和陷阱

虽然思维链很强大，但它也有局限。

可能生成错误的推理

模型可能生成看起来合理，但实际错误的推理过程。

这里不给出示例，因为简单的推理现在大模型基本不会出错。但你要知道，在某些情况下，推理是有可能出错的。毕竟我们之前已经学习过，大模型的回答是具有随机性的。

增加了Token消耗

对比：

> 标准回答： “答案：21杯” ← 3个tokens 思维链回答： “让我一步步思考： 咖啡店原来有23杯，上午卖了7杯，剩下23-7=16杯。 下午又做了5杯，所以16+5=21杯。 答案：21杯” ← 约40个tokens

消耗增加了10倍以上。如果使用API,这意味着成本增加10倍。

对简单问题可能过度复杂化

> 问题:“2+2等于多少?” 思维链： “让我仔细思考： 2是一个数字，另一个2也是一个数字。 加法意味着将两个数字相加。 2+2表示将2和2相加。 1+1=2,2+1=3,2+2=4。 所以答案是4。” ← 过度复杂化，浪费 直接答: “4” ← 完全够用

现在很多大模型，比如DeepSeek，都有深度思考模式。但其实有时候，简单的问题，你无需开启深度思考模式，这个模式速度很慢。

8.6 冷知识与趣闻

趣闻一：思维链是意外的发现

CoT的诞生其实是个“美丽的意外”。

2022年，谷歌研究团队在测试GPT-3时，某位研究者在准备示例时“习惯性地”多写了几行推理步骤，想着“这样示例更清晰”。结果跑实验时发现，准确率突然从17%跳到了57%！

他们起初以为是实验出错了，反复验证后才确认：原来在示例里展示推理过程，能让模型"学会"如何一步步思考。这个发现直接催生了一篇引用量超过7000次的经典论文。

更有趣的是，后来又有人发现，甚至不给示例，只要加一句"Let’s think step by step"，模型也会自动展示推理——这就是后来的Zero-Shot CoT。从"在示例里多写几行"到"只加一句话就能触发推理"，这个进化路径本身就很神奇。

趣闻二：大模型也会"边说边改"

使用CoT时，有个让人惊讶的现象：模型在生成推理过程中，可能突然"发现"自己算错了，然后主动改口纠正。

早期典型的例子是这样的：

> 问题：小明有10个苹果，吃了3个，又买了5个，现在有多少？ 模型推理： 小明原来有10个苹果 吃了3个，剩下 10 - 3 = 7个 又买了5个，所以 7 - 5... 等等，这里应该是加法！ 7 + 5 = 12个 答案：12个

这个"等等"让很多研究者第一次看到时都吓了一跳——模型居然会在推理中途"意识到"错误并自我修正，就像人类"边说边想"时突然发现不对劲。

更神奇的是，这种自我纠错不是研究者刻意训练的，而是CoT自然“涌现”出来的能力。当模型被要求展示推理步骤时，它生成的每一步都会成为后续推理的“输入”，这让模型有机会“看到”前面的错误并纠正。这也解释了为什么CoT能显著提升准确率——不仅因为分步推理更简单，还因为模型获得了“边想边改”的机会。

8.7 下节预告

在本节中，我们看到通过Chain-of-Thought让模型“边想边说”,可以大幅提升复杂任务的准确率，也让推理过程变得可检查、可审查。

下一节，我们将把视角从“如何思考”转向“思考的风格”,看看Temperature 和 Top-p 这些采样参数如何决定答案是保守还是发散、稳定还是惊喜。

8.8 推荐阅读

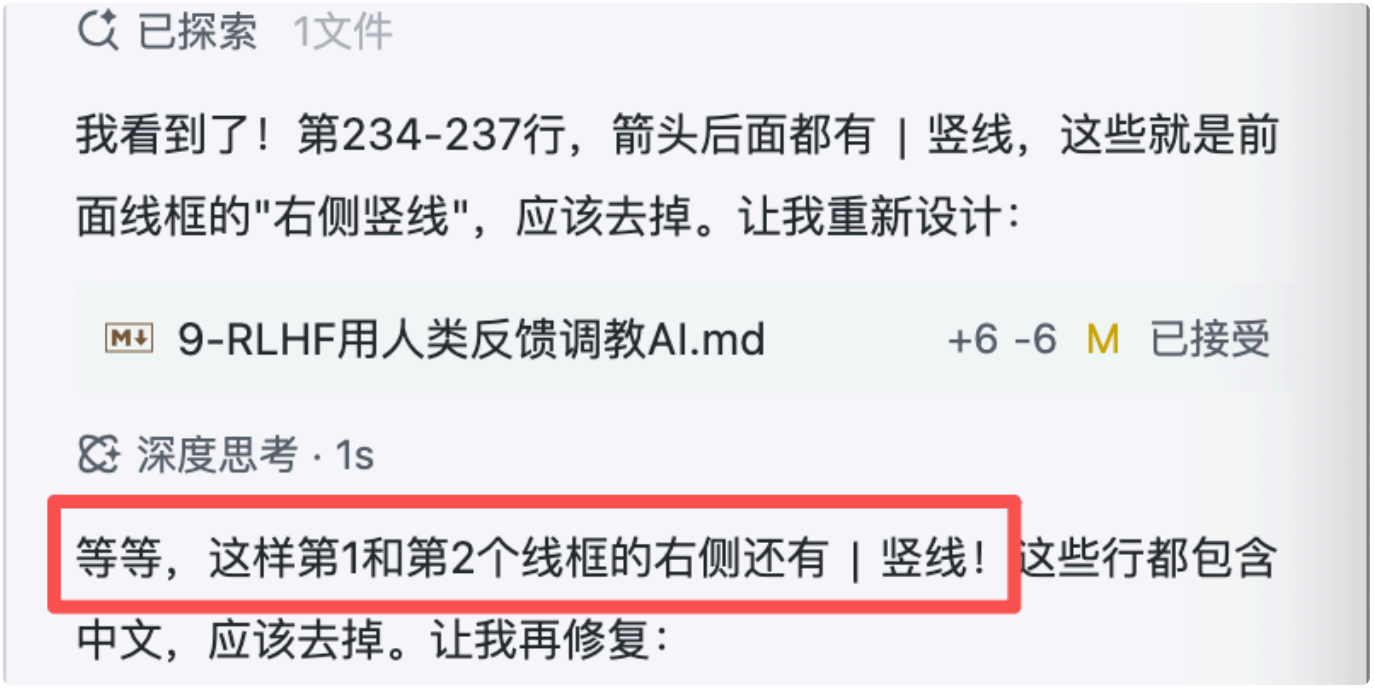
Chain-of-Thought Prompting Elicits Reasoning in Large Language Models (Wei 等， 2022)

系统性地提出并评估了CoT提示方法，展示了在数学、符号推理等多种任务上，仅通过在提示中加入推理步骤就能显著提升大模型的推理能力，是理解本节内容的关键论文。

8.9 ■ 学点英语

中文	English	音标	说明
思维链	Chain-of-Thought (CoT)	/tʃeɪn ɒv θɔːt/	让模型显式输出中间推理步骤的提示技术
分步推理	Step-by-step Reasoning	/step baɪ step ˈriːzənɪŋ/	把复杂问题拆为可逐步推导的小问题
自回归	Autoregressive	/ˌɔːtəʊrɪˈgresɪv/	根据已生成内容预测下一个 Token 的生成方式
推理	Reasoning	/ˈriːzənɪŋ/	基于前提推导结论的思维过程
自我纠错	Self-correction	/ˌself kəˈrekʃn/	模型在后续步骤中修正前面错误的能力
工作记忆	Working Memory	/ˈwɜːkɪŋ ˈmeməri/	临时存放中间计算结果的内在缓存
算力预算	Compute Budget	/kəmˈpjuːt ˈbʌdʒɪt/	单次推理允许消耗的最大 Token 或计算量
阈值效应	Threshold Effect	/ˈθreʃhəʊld ɪˈfekt/	性能收益只在规模跨过某一点后才出现的非线性现象

图片


已探索 1文件
我看到了！第234-237行，箭头后面都有 | 竖线，这些就是前面线框的"右侧竖线"，应该去掉。让我重新设计：
9-RLHF用人类反馈调教AI.md +6 -6 M 已接受
深度思考 · 1s
等等，这样第1和第2个线框的右侧还有 | 竖线！这些行都包含中文，应该去掉。让我再修复：